

any list objects inside the list object being copied (and any list objects in those list objects, and so on). Enter the following into the interactive shell:

---

```
>>> import copy
>>> bacon = [[1, 2], [3, 4]]
>>> ham = copy.deepcopy(bacon)
>>> id(bacon[0]), id(ham[0])
(2356896337352, 2356896466184)
>>> bacon[0][0] = 'CHANGED'
>>> bacon
[['CHANGED', 2], [3, 4]]
>>> ham
[[1, 2], [3, 4]]
```

---

A visualization of the execution of this code is at <https://autbor.com/copydeepcopy>. Although `copy.deepcopy()` is slightly slower than `copy.copy()`, it's safer to use if you don't know whether the list being copied contains other lists (or other mutable objects like dictionaries or sets). My general advice is to always use `copy.deepcopy()`: it might prevent subtle bugs, and the slowdown in your code probably won't be noticeable.

## Don't Use Mutable Values for Default Arguments

Python allows you to set *default arguments* for parameters in the functions you define. If a user doesn't explicitly set a parameter, the function will execute using the default argument. This is useful when most calls to the function use the same argument, because default arguments make the parameter optional. For example, passing `None` for the `split()` method makes it split on whitespace characters, but `None` is also the default argument: calling `'cat dog'.split()` does the same thing as calling `'cat dog'.split(None)`. The function uses the default argument for the parameter's argument unless the caller passes one in.

But you should never set a *mutable* object, such as a list or dictionary, as a default argument. To see how this causes bugs, look at the following example, which defines an `addIngredient()` function that adds an ingredient string to a list that represents a sandwich. Because the first and last items of